

第四章：地址空间

引言

本章导读

本章展现了操作系统一系列功能：

1. 动态内存分配：操作系统允许程序在运行时按需申请和释放内存，而不是一开始就分配固定大小的内存。这样可以让多个程序更灵活、高效地使用有限的物理内存。
2. 页表的虚实内存映射机制简化编译器工作：有了虚拟内存，编译器在生成程序时不需要关心程序最终会加载到物理内存的哪个位置。它只需要假设程序从某个固定的虚拟地址开始运行，操作系统通过页表自动把虚拟地址转换成真实的物理地址。
3. 页表加强内存隔离与系统安全：每个程序都有自己的虚拟地址空间，彼此之间不能随意访问对方的内存；同时，应用程序也无法直接访问操作系统内核的内存。这种隔离靠页表来实现，能有效防止程序出错或恶意行为破坏整个系统。
4. 页表支持“空分复用”（Spatial Multiplexing）的概念：意思是多个程序可以“同时”存在于内存中（哪怕物理内存不够），通过虚拟内存机制让它们各自以为自己拥有独立、完整的地址空间。虽然书中提到这个想法，但暂时还没真正实现。

上一章，我们分别实现了多道程序和分时多任务系统，它们的核心机制都是任务切换。

1. 多道程序的目标是提高 CPU 利用率：当一个程序在等待 I/O 时，CPU 可以切换去运行另一个程序。
2. 分时多任务的目标是让用户感觉多个程序在“同时”运行，通过快速轮流分配 CPU 时间片来实现。

尽管两者设计目标不同，导致任务切换的时机和策略不同，但有一个共同点：任务切换对应用程序是完全透明的（Transparent）。

这意味着：

1. 应用程序不需要知道操作系统什么时候会把它暂停、切换到别的程序；
2. 应用也不需要为任务切换做任何特殊处理（除非开发者想做性能优化）；
3. 甚至，应用完全可以“以为”自己一直在独占 CPU 运行。

事实上，在大多数应用（或者说应用开发者）看来，它们好像独占了整个 CPU，并且拥有一块专属的内存空间（这块内存可能是连续的，也可能不连续，但对程序来说看起来是连续的）。

但我们知道，这其实是一种幻象（Illusion）：

1. CPU 并没有被某个程序独占，而是被多个程序轮流使用；
2. 操作系统通过时分复用（TDM, Time-Division Multiplexing）把 CPU 时间切分成小片段，分给不同程序；
3. 而这一切都被操作系统巧妙地隐藏起来，应用程序看不到底层的真实情况，只看到一个“干净、独占”的运行环境。

我们现在的内存管理方法很粗糙，就是把程序简单塞进物理内存的某个地方。

具体来说，程序运行时，是直接使用物理地址去读写内存的，这就导致程序员必须清楚自己的程序会被放在内存的哪个具体位置。另外，为了便于切换任务，我们在上一章的做法是：在系统启动时，就把所有程序一次性全部加载到内存里，并且它们会一直待在原地，直到所有程序都运行完毕。这种直接使用物理地址的做法，带来了三个主要问题：

1. 内核给程序的内存接口不好用，也不“透明”。

因为程序直接操作物理内存，它在编写时就得预先确定自己应该被加载到哪个物理地址上。如果同时有多个程序，为了避免它们占用同一个内存位置，可能还需要程序员之间互相商量、分配地址。这在今天看来，是极其麻烦且不合理的事情。

2. 内核没有对程序的内存访问行为进行任何保护，每个程序都能读写整个物理内存。

即使程序被限制在较低的“用户特权级”下运行，它依然能惹出大麻烦：例如，它可以读写其他程序的数据，从而窃取信息或破坏其运行；更严重的是，它甚至可以直接修改内核的代码，比如替换掉关键的异常处理程序，从而挟持内核、执行恶意代码。总之，这让整个系统既不安全，也不稳定。

3. 程序能使用的内存空间在运行前就固定死了，内核无法灵活地提供动态内存。

比如，当一个程序运行结束后，它占用的内存空间就被释放了，但这部分空间无法被动态地分配给其他还在运行的程序使用。

因此，为了防止程序“胡作非为”，本章我们将实现更好的物理内存管理，并为程序提供一个抽象出来的、更透明易用、也更安全的访问内存的接口，这就是基于分页机制的虚拟内存。

从程序的角度看，虚拟内存的效果就是：它仿佛拥有一个从“0”地址开始的、非常大的、可以读/写/执行的连续地址空间。

实施地址空间的首要步骤是引入分页机制，完成虚拟内存与物理内存之间的页面映射。这一过程需要处理硬件层面的具体信息，涉及多种不同的地址映射配置，因此较为复杂。总的来说，我们需要考虑以下几个关键问题：

- 硬件所提供的物理内存具体范围是怎样的？
- 哪些部分的物理内存需要建立页映射？
- 应如何建立页表以启用分页功能？
- 怎样保证操作系统在启用分页机制前后都能够正确寻址并顺利执行代码？
- 页目录表（一级页表）的起始地址应设置在何处？
- 二级、三级等各级页表的起始地址应如何安排，各自需要多少空间？
- 页目录表项的内容应如何设置？
- 其他各级页表项的内容又该如何配置？
- 如果要为每个任务提供独立的地址空间，是否每个任务都需要拥有专属的页表？
- 代表用户程序的任务与操作系统是否需要各自独立的页表？
- 在启用页表之后，任务与操作系统之间应通过什么方式传递数据？

若能妥善解决以上问题，我们就能够设计并实现一个具备高度安全防护能力的“头甲龙”操作系统。同时，也能更深入地理解地址空间、虚拟地址等操作系统抽象概念，及其与实际虚拟内存实现之间的关联。

多Git仓库管理指南

针对存在多个远程代码仓库的场景，可通过 `git remote add origin url` 指令将另一个远程仓库添加为remote配置。此处的url参数需填写目标远程仓库的访问地址。

运行 `git remote -v` 命令可以列出当前本地已关联的所有远程仓库。若要移除某个远程仓库关联，可使用 `git remote rm origin` 指令。

需要注意的是，origin是对远程仓库设置的别名，该名称并非固定不变，用户可根据需要自行设定。建议为不同的远程仓库配置不同的别名，以便在执行push、pull等操作时能够清晰区分目标仓库。

后续在执行git push、git fetch、git pull等常用命令时，原先使用的默认名称origin需根据实际情况替换为对应的远程仓库别名。本次实验的代码库同时存在于GitHub和GitLab平台。如需从GitHub仓库获取代码，可参照上述方法配置新的GitHub远程仓库。

```
$ cd uCore-Tutorial-Code-2023S
# 你可以将 upstream 改为你喜欢的名字
$ git remote add upstream https://github.com/LearningOS/uCore-Tutorial-Code-2023S.git
# 更新仓库信息
$ git fetch upstream
# 查看已添加的远程仓库：应该能看到已有一个 origin 和新添加的 upstream 仓库
$ git remote -v
# 根据需求选择以下一种操作即可
# 在本地新建一个与远程仓库对应的分支：
$ git checkout -b ch4 upstream/ch4
# 本地已有分支，从远程仓库更新：
$ git checkout ch4
$ git merge upstream/ch4
# 将更新推送到自己的远程仓库
$ git push origin ch4
```

实践体验

本章的应用和上一章相同，只不过由于内核提供给应用的访存接口被替换，应用的构建方式发生了变化，这方面在下面会深入介绍。因此应用运行起来的效果与上一章是一致的。

获取本章代码：

```
$ git checkout ch4
```

在 qemu 模拟器上运行本章代码：

```
$ make test BASE=1
```

本章代码树

```
.
├── bootloader
│   └── rustsbi-qemu.bin
├── LICENSE
├── Makefile
├── os
│   ├── console.c
│   ├── console.h
│   ├── const.h
│   ├── defs.h
│   ├── entry.S
│   ├── kalloc.c (新增，物理页帧分配器)
│   ├── kalloc.h (新增，物理页帧分配器头文件)
│   ├── kernel.ld
│   ├── kernelld.py
│   ├── loader.c
│   ├── loader.h
│   ├── log.h
│   └── main.c
```

```
|   ├── pack.py
|   ├── printf.c
|   ├── printf.h
|   ├── proc.c
|   ├── proc.h
|   ├── riscv.h
|   ├── sbi.c
|   ├── sbi.h
|   ├── string.c
|   ├── string.h
|   ├── switch.S
|   ├── syscall.c
|   ├── syscall.h
|   ├── syscall_ids.h
|   ├── timer.c
|   ├── timer.h
|   ├── trampoline.S
|   ├── trap.c
|   ├── trap.h
|   ├── types.h
|   ├── vm.c (新增, 虚拟内存管理)
|   └── vm.h (新增, 虚拟内存管理头文件)
└── README.md
└── scripts
    ├── kernelld.py
    └── pack.py
└── user
```

本章代码导读

本章涉及的代码量相对较大，主要集中在页表处理相关部分。建议同学们在学习过程中结合操作系统课程中有关页表的内容进行回顾和理解。此外，本章将详细介绍本操作系统在 Riscv-64 指令集下页表的设计方式，以及页表读取与修改的具体实现方法。

C 语言中的动态内存分配

本节导读

目前，若将内核视为一个应用，其中所有变量均在内存中静态分配，这种方式在空闲内存的使用上缺乏灵活性。因此，操作系统需要提供动态申请和释放内存的能力，以增强其对各类基于内存的资源分配与管理能力。

从应用程序的视角看，动态内存分配所使用的内存区域即为操作系统管理的“堆（Heap）”。而在操作系统内部，需要自身具备动态内存分配的功能。为实现这一能力，操作系统需包含以下功能：

1. 在初始阶段提供一块较大的内存空间作为初始的“堆”。在不启用分页机制的情况下，该空间为物理内存空间；否则，其为虚拟内存空间。
2. 提供在堆上分配内存的函数接口，使得调用方能够获得一块地址连续的空闲内存区域进行读写操作。
3. 提供释放内存的函数接口，实现对内存的回收，以支持后续的内存分配请求。
4. 实现连续内存分配算法，对空闲内存空间进行有效管理，从而动态维护一系列已分配和未分配的内存块。
5. （可选）提供基于堆的数据结构及相关操作。基于上述内存分配与释放接口，可实现如动态数组、动态字典等空间大小可灵活调整的堆数据结构，进一步提升编程的灵活性。

在使用 C++ 语言时，同学们对 `new/delete` 的使用方法应已较为熟悉。而在 C 语言中，动态内存的申请通过以下函数实现：

```
void* malloc (size_t size);
void free (void* ptr);
```

在实验过程中，`malloc` 函数用于从堆区动态分配一块指定大小的内存空间，其大小参数为 `size`（单位为字节）。该函数会返回一个指向所分配内存起始地址的指针。当该内存区域不再使用时，应调用 `free` 函数并将该指针作为参数传入，以便系统回收对应的堆内存。

值得注意的是，在程序中我们仅能通过返回的指针变量间接访问堆上的内存空间，而无法直接对其进行操作。实际上，程序中所有直接可见的变量均被静态分配在栈或全局数据段中，它们的大小在编译期间即已确定。例如，指针类型的大小通常等于计算机可寻址空间的位宽。这些变量能够作为背后一块在编译时无法确定大小的内存区域的“代表”，这一机制在内存管理中具有重要意义。

针对物理内存的管理，同一页内的虚拟地址所对应的物理内存是连续的。然而，连续的虚拟地址空间并不一定对应连续的物理地址空间。因此，我们需要一种数据结构来记录哪些物理内存页当前处于可用状态。为管理这种非连续的内存分布，本实验采用链表结构进行组织：将每个大小为 `PAGE` 的空闲物理内存块视为一个链表节点（`listnode`），并通过链表实现对物理内存的分配与回收管理。

上述相关功能的实现代码位于 `kalloc.c` 文件中。

kalloc之中的动态内存分配

我们采用链表结构记录空闲的物理地址。因此当应用程序申请一段动态内存的时候，只需要把链表头所指向地址拿出即可。

```
// os/kalloc.c
/* kalloc - 分配一个4096字节的物理内存页面
 * 返回值：
 *   成功：返回指向分配页面的内核虚拟地址指针
 *   失败：返回 NULL
 */
void *kalloc(void)
{
    struct linklist *l;

    // 从空闲链表头部获取一个可用页面
    l = kmem.freelist;

    if (l) {
        // 摘除当前页面：将空闲链表头移到下一个节点
        kmem.freelist = l->next;

        // 调试用：填充为5，检测未初始化内存的使用错误
        memset((char *)l, 5, PGSIZE);
    }

    return (void *)l;
}

/*
```

```

* kfree - 释放一个物理内存页面
* @pa: 要释放的物理页起始地址（必须是页对齐的）
* 功能：将释放的页面重新插入到空闲链表中
*/
void kfree(void *pa)
{
    struct linklist *l;

    // 安全检查：地址必须页对齐且在合法物理内存范围内
    if (((uint64)pa % PGSIZE) != 0 ||
        (char *)pa < ekernel ||
        (uint64)pa >= PHYSTOP) {
        panic("kfree"); // 非法地址，触发内核panic
    }

    // 调试用：填充为1，捕获use-after-free错误
    memset(pa, 1, PGSIZE);

    // 将释放的页面插入空闲链表头部
    l = (struct linklist *)pa;
    l->next = kmem.freelist;
    kmem.freelist = l;
}

```

我们在 `main` 函数中会执行 `kinit`，它会初始化从 `ekernel` 到 `PHYSTOP` 的所有物理地址作为空闲的物理地址。`freerange` 中调用的 `kfree` 函数以页为单位向对应内存中填入垃圾数据（全1），并把初始化好的一个页作为新的空闲 `listnode` 插入到链表首部。

注意：C语言之中要求进行内存回收，也就是 `malloc` 以及 `free` 要成对出现。但是我们的OS中不强制要求这一点，也就是如果测例本身未在申请动态内存后显式地调用 `free` 来释放内存，OS无需帮助它释放内存。

地址空间

本节导读

到目前为止，我们为应用程序提供的内存视图是非常原始的物理内存空间，可以将其简单理解为一个可以任意读写的连续数组。为了限制应用程序对内存的访问范围，并为操作系统提供灵活的内存管理手段，计算机硬件引入了多种内存保护与地址映射机制，例如 RISC-V 中的基址-边界翻译与保护机制、x86 架构中的分段机制，以及 RISC-V/x86/ARM 等架构普遍支持的分页机制。这些机制的共同点是：CPU 在执行程序时所访问的指令和数据所使用的地址是虚拟地址，必须通过硬件机制转换为合法的物理地址，否则将触发访问异常。

为了充分发挥这类硬件机制的作用，操作系统必须扩展和增强自身的管理能力。操作系统不仅需要妥善管理物理内存与虚拟内存，还需向应用程序提供统一的访问接口——使得应用程序无需关心底层使用的是虚拟内存还是物理内存。为此，操作系统引入了地址空间（Address Space）这一抽象概念，并在内核中建立虚拟地址到物理地址的映射机制，从而为应用程序营造一个透明、一致的虚拟内存环境。

本节将结合操作系统的发展历程，回顾地址空间抽象的实现策略是如何逐步演变的。

虚拟地址与地址空间

地址虚拟化出现之前

在早期计算机系统中，硬件资源被用来直接运行单个“裸机”应用程序。此时，系统尚未形成真正意义上的操作系统，仅可视为一个向应用提供基础功能（如输入/输出）的函数库。从内存使用角度看，物理内存的一部分被用来存放该函数库的代码与数据，剩余的全部物理内存空间则完全交由该应用程序独立支配。应用程序可依据自身需求，将所使用的内存划分为若干功能区域，例如：存放程序指令的代码段、存放全局变量的全局数据段、用于动态内存分配的堆以及用于函数调用与局部变量的栈等。由于整个系统中只有一个应用在运行，因此它如何规划和使用这些内存区域完全由其自身决定，不受其他因素干扰。

随后的批处理系统在内存使用模式上与上述裸机应用较为相似。其主要区别在于：批处理系统会按顺序依次加载并运行多个应用程序。在任一时刻，正在运行的应用同样独占除操作系统内核占据区域之外的全部物理内存。当一个应用程序运行结束（或发生错误退出）后，操作系统会清空它所占据的内存区域，再将序列中的下一个应用程序加载到相同的物理内存区域中运行。因此，对于每个应用程序的开发者而言，他们看到的“内存世界”是一致的且确定的：他们都假设自己的程序会被加载到一块固定的、起始地址已知的物理内存中运行，并基于此认知来设计程序的内存布局（例如，变量存放在哪个地址，代码从何处开始执行）。

为了提升CPU的利用率（避免CPU因等待慢速的I/O操作而闲置），计算机系统后来发展出了多道程序技术。更进一步，为了提供更好的交互体验，支持多个用户同时使用，分时多任务系统应运而生。这些系统的关键特征是：应用程序不再一直运行直到结束，而是会进入一种“暂停”状态。暂停可能由应用程序主动调用 `yield` 等操作让出CPU引起，也可能由操作系统内核在经过一段设定的时间后强制剥夺其CPU使用权（即“时间片用尽”）导致。

这就引出了一个核心问题：当一个应用程序被“暂停”时，它存放在内存中的代码和数据应该如何处理？早期的解决方案之一是延续批处理系统的思路：每个应用在运行时依然独占内核之外的全部物理内存。当它被暂停时，操作系统需要将其全部的代码和数据完整地保存到磁盘（或其它外部存储设备）上；随后，再将下一个要运行的应用从磁盘上完整地加载到内存的同一个区域。这个过程完成后，新的应用才能开始执行。

然而，这种方案的开销极大，几乎不可行。原因在于：内存与磁盘之间的数据传输速度，相比CPU的处理速度要慢好几个数量级。每次任务切换都要进行如此大量的数据搬运，会导致切换过程极其缓慢，CPU大部分时间都在等待数据搬运，严重降低了系统整体效率。

因此，更可行的方案出现了：让多个应用程序的代码和数据同时驻留在物理内存中。为了实现这一点，操作系统需要限制每个应用程序可使用的最大内存空间，确保所有应用的内存需求总和不超过物理内存的总容量（实际上会留出一部分给内核）。这样，在任务切换时，操作系统就无需进行耗时的内存与磁盘间的数据交换，而只需完成任务上下文的保存与恢复。这里的“上下文”主要指CPU中少数几个通用寄存器的值（如程序计数器PC、栈指针SP等），这些操作完全在高速的内存中进行，速度极快，从而极大地降低了任务切换的开销。这也是我们在第三章中实现多道程序系统时所采用的方法。

但第三章的做法也给应用程序开发带来了新的挑战和困难，主要体现在以下两方面：

- 1.对应用开发者不友好：应用程序需要预先确定自己将被加载到哪个具体的物理地址运行，并且直接访问真实的物理地址。这就要求开发者必须深入了解硬件内存布局，增加了额外的学习成本，也给程序的编写、移植和调试带来了极大的不便。不同物理机器上的内存情况可能不同，为一种环境编写的程序可能无法在另一种环境中直接运行。
- 2.对操作系统（内核）管理不利：内核将直接访问物理内存的权力完全下放给应用程序，使得自身难以对应用程序的访存行为进行有效监管和控制。即使拥有特权级机制，操作系统也无法防止应用程序的恶意或错误行为，例如：应用程序可能有意或无意地访问、修改属于操作系统内核或其他应用程序的内存数据，从而破坏系统安全性与稳定性。

为了解决上述问题，实现更高效、更安全、更易于开发的内存管理，地址虚拟化技术便被引入，其核心思想是为每个应用程序创建一个独立的、统一的、从零开始的虚拟地址空间。下文将详细介绍这一概念。

通过抽象层增强内存管理

为应对上述难题，抽象仍是核心指导原则。此处的抽象指内核需统一管理物理内存，并向上层应用程序提供一组抽象接口。结合此前经验，该抽象层需满足以下设计目标：

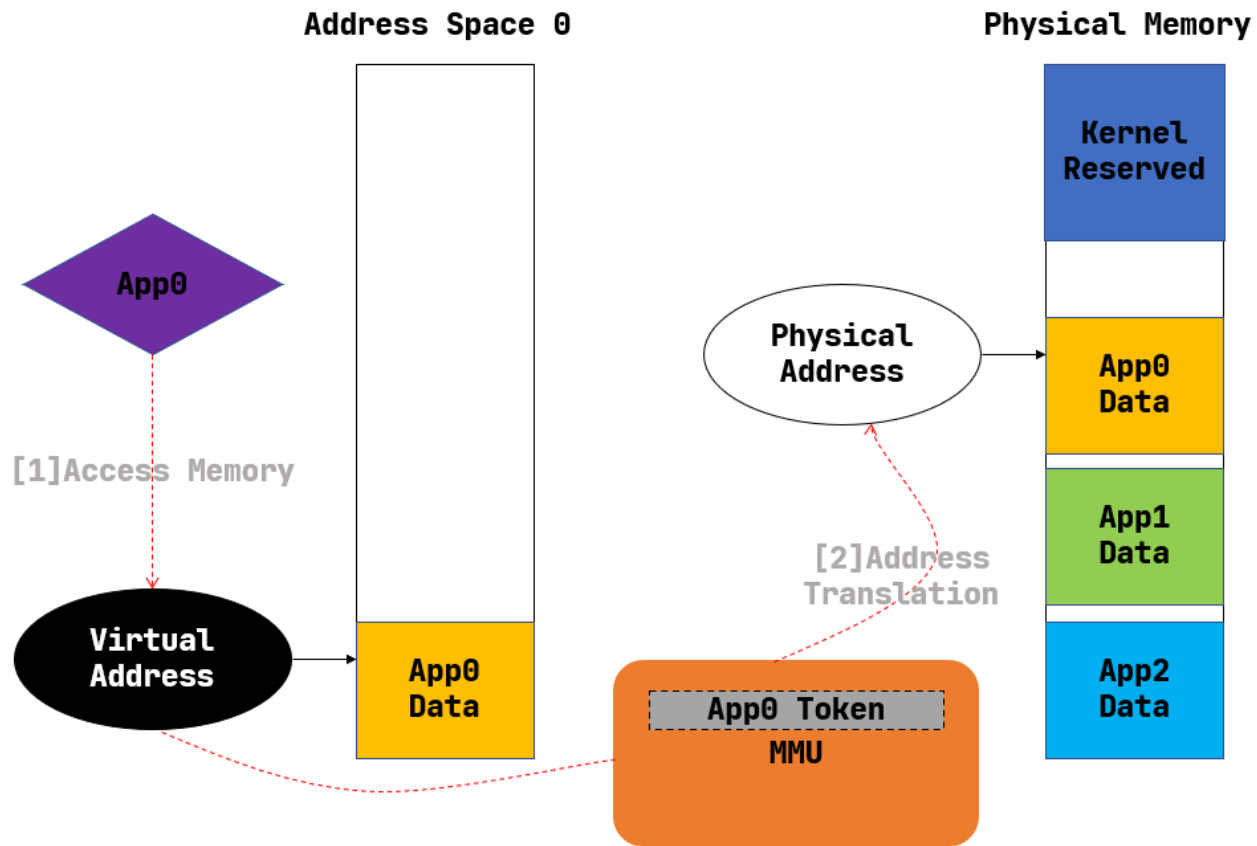
- 1. 透明性：应用开发者无需关注底层物理内存的硬件细节，且在一般情况下也不必了解内核的具体实现策略，从而降低其认知负担；
- 2. 高效性：该抽象在多数情况下不应引入显著性能开销；
- 3. 安全性：该抽象应能有效识别并阻止应用程序越界访问其他应用或内核的代码与数据等恶意行为。

至今仍被多数操作系统内核广泛采用的抽象称为 地址空间（Address Space）。可将其理解为一个庞大但不一定实际存在的内存区域。在每个应用程序看来，操作系统会为它分配一块连续、独占且范围有限（但实际上足够大）的地址空间。该空间中有部分区域受到操作系统限制（例如内核自身占用的虚拟地址空间），应用程序不能随意访问。除此之外，应用程序可在允许的地址范围内自由规划内存布局，将其各段（如代码段、数据段、堆栈段等）放置在自身地址空间的任意合法位置。应用程序也可通过地址来读写自身地址空间中的数据，如同使用物理地址访问物理内存一样。这类地址称为 虚拟地址（Virtual Address）。

为实现地址空间抽象，操作系统需要硬件支持，即计算机组成原理课程中所介绍的 MMU（内存管理单元）与 TLB（转址旁路缓存）等硬件机制。

自此，应用程序能够直接访问的内存仅限于操作系统为其分配的地址空间，且其所有访存操作均使用虚拟地址，包括取指令、读写栈、堆及全局数据段等。实际上，通过扩展特权级机制，应用程序已失去直接通过物理地址访问物理内存的权限。应用程序的执行环境在安全性上得到进一步加强，形成了 用户态特权级与地址空间双重保护机制。

由于每个应用程序独享一个地址空间，其中仅包含自身的各个段，因此可自由安排各段位置，无需考虑与其他应用冲突；同时，它也无法窃取或破坏其他应用的数据，因为这些数据位于其他应用的地址空间中，而应用程序只能通过虚拟地址访问自身的地址空间。这体现了地址空间抽象对应用程序执行安全性与稳定性的保障。



应用程序的数据最终仍存储在物理内存中，那么虚拟地址如何构成地址空间？虚拟地址空间又如何映射到物理内存呢？操作系统可通过设计精巧的数据结构来表示地址空间。然而，若完全由操作系统处理每次处理器访问所涉及的虚实地址转换，开销将非常巨大。因此，需要硬件功能扩展以加速地址转换过程（可回顾计算机组成原理中介绍的 MMU 与 TLB）。

引入硬件加速虚实地址转换

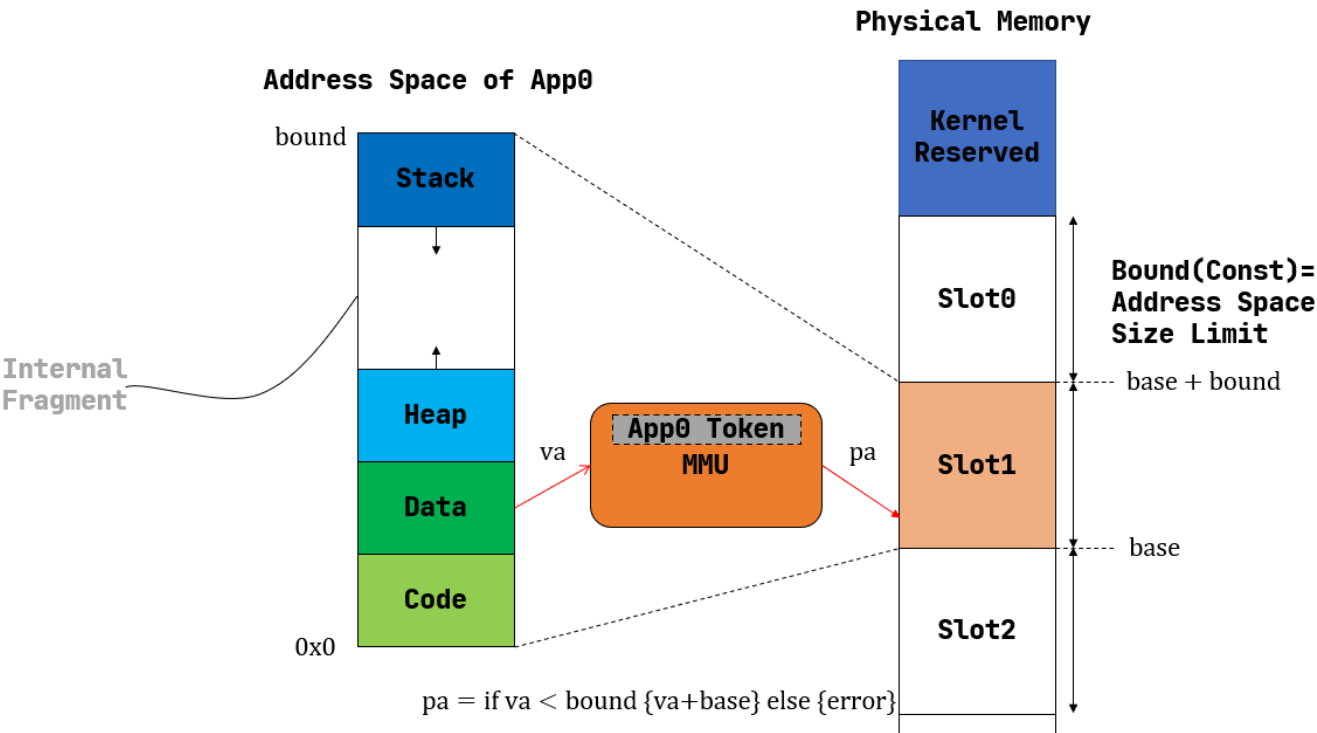
回顾计算机组成原理知识：如上图所示，当应用程序执行取指令或访存指令时，均以虚拟地址作为索引访问自身地址空间。此时，CPU 中的 内存管理单元（MMU） 自动将该虚拟地址通过地址转换（Address Translation）转换为物理地址，即数据实际存放的物理内存位置。也就是说，在 MMU 的协助下，应用程序对自身地址空间的访问才能被实际映射为对物理内存的访问。

实际上，每个应用程序的地址空间均可视为一套从虚拟地址到物理地址的映射关系。不同应用程序的映射可能不同，即 MMU 可能将来自不同应用程序地址空间的相同虚拟地址转换为不同的物理地址。为实现这一点，硬件提供若干可配置的寄存器，软件可通过设置这些寄存器来控制 MMU 按哪个应用程序的地址空间进行地址转换。因此，内核作为软件部分，需要完成以下工作：将应用程序数据放置在物理内存并进行管理，同时在任务切换时，同步切换那些控制 MMU 地址映射的寄存器。

回顾此前对 CPU 资源抽象——时分复用的介绍：该机制为应用程序营造了“每个应用独占整个 CPU”的假象，隐藏了多个应用分时共享 CPU 的实际状况。地址空间抽象同样如此：应用程序只需且只能看到它“独占整个地址空间”的表象，而背后的实质仍是多个应用程序共享物理内存，它们的数据分别存储于内存的不同位置。

地址空间仅为一层抽象接口，其具体实现策略多样。不同的实现策略会影响操作系统内核如何安排应用程序数据在物理内存中的存放位置，以及 MMU 如何进行地址转换。接下来，我们将简要介绍几种历史上曾采用的策略，并分析其优缺点。

分段内存管理



早期的内存管理方式如上图所示：每个应用程序被限制使用固定大小的地址空间，这个大小设为常数 `bound`，因此每个应用可用的虚拟地址范围都是从 `0` 到 `bound`。然后，物理内存除了内核使用的部分外，被划分为多个大小相同的“插槽”（Slot）。每个应用程序的所有数据都会被内核放在其中一个插槽里，对应一段连续的物理地址。假设插槽的起始物理地址是 `base`，由于插槽和地址空间的大小相同，实际物理地址范围就是 `[base, base + bound)`。地址转换

也很简单：只要检查虚拟地址不超过 `bound` 的限制（超过则会触发异常），然后将其加上 `base` 就得到了真正的物理地址。

这种方式实现起来非常简单：内存管理单元（MMU）只需要 `base` 和 `bound` 两个寄存器，在地址转换时进行比较或加法运算即可。而内核在切换任务时，只需要切换 `base` 寄存器（因为 `bound` 是固定的）。内存管理方面，内核只需用一个位图（Bitmap）来记录哪些插槽被占用，应用启动或退出时更新位图即可。

然而，这种方式浪费了大量内存。因为每个应用的地址空间都需要预留一部分，用来让栈向低地址增长，堆向高地址增长（以便支持动态内存分配）。不同应用的内存需求各不相同，内核只能按照内存需求最高的应用来统一设置地址空间的大小。这样一来，内存需求较低的应用就无法充分利用分配给它们的空间，而这片未使用的空间又无法被其他应用使用，它仍然占据整个插槽的一部分。这种在地址空间内部无法被充分利用的空间被称为“内碎片”（Internal Fragment）。内碎片限制了系统能同时运行的应用数量。无论内核如何设置地址空间的大小，都难以适应多样化的应用需求。这就是固定参数的缺陷：实现简单，但缺乏灵活性。

为了解决这个问题，人们提出了分段管理策略，如下图所示：

分段管理不再以整个应用的地址空间为单位，而是以更细的粒度——即地址空间中的各个逻辑段（如代码段、数据段、堆段、栈段）为单位，来安排数据在物理内存中的位置。每个段在虚拟地址空间和物理内存中的映射关系是不同的，因此MMU需要为每个段分别使用一对 `base` 和 `bound` 寄存器。由于每个段的大小不同，`bound` 也不再是固定的。在任务切换时，这些寄存器也需要跟着切换。

为了简化，这里我们忽略一些细节，比如应用在访问某个虚拟地址时，硬件如何判断它属于哪个段并选择正确的 `base/bound` 寄存器进行检查和转换。我们重点关注分段管理是否解决了内碎片带来的内存浪费问题。可以看到，每个段在物理内存中只占用与实际使用大小相等的空间。堆比较特殊，它的大小可能在运行时增长，但这需要应用通过系统调用向内核申请。也就是说，这是一种按需分配，而不是一开始就给每个应用分配一大块可能用不完的内存。因此，内碎片的问题就被解决了。

尽管内碎片消失了，但内存浪费的问题并没有完全解决。因为每个段的大小可能不同，内核需要使用更通用但也更复杂的“连续内存分配算法”来管理内存，而不能像之前那样简单地按固定大小的插槽分配。连续内存分配算法每次需要找到一块连续的物理内存来存放一个段的数据。随着内存不断分配和释放，物理内存中会留下一些不连续的小块空闲区域，其中有些是两个已分配块之间的小间隙，它们可能因为太小而无法被再次分配使用，这些被称为“外碎片”（External Fragment）。

如果此时要分配一个较大的内存块，就需要把这些分散的外碎片“拼凑”成一个大的连续块。然而，这样的操作开销很大，涉及大量的内存读写，需要移动和调整已有的内存块，才能让小碎片合并成大块空闲内存。如果选择合适的连续内存分配算法，可以减少这种情况的发生。常见的算法包括首次适应、最坏适应、最佳适应和伙伴系统等，它们各有优缺点，具体表现取决于实际的应用需求。

那么，分段内存管理所带来的外碎片问题，以及连续内存分配算法较复杂的问题，是否有办法解决呢？

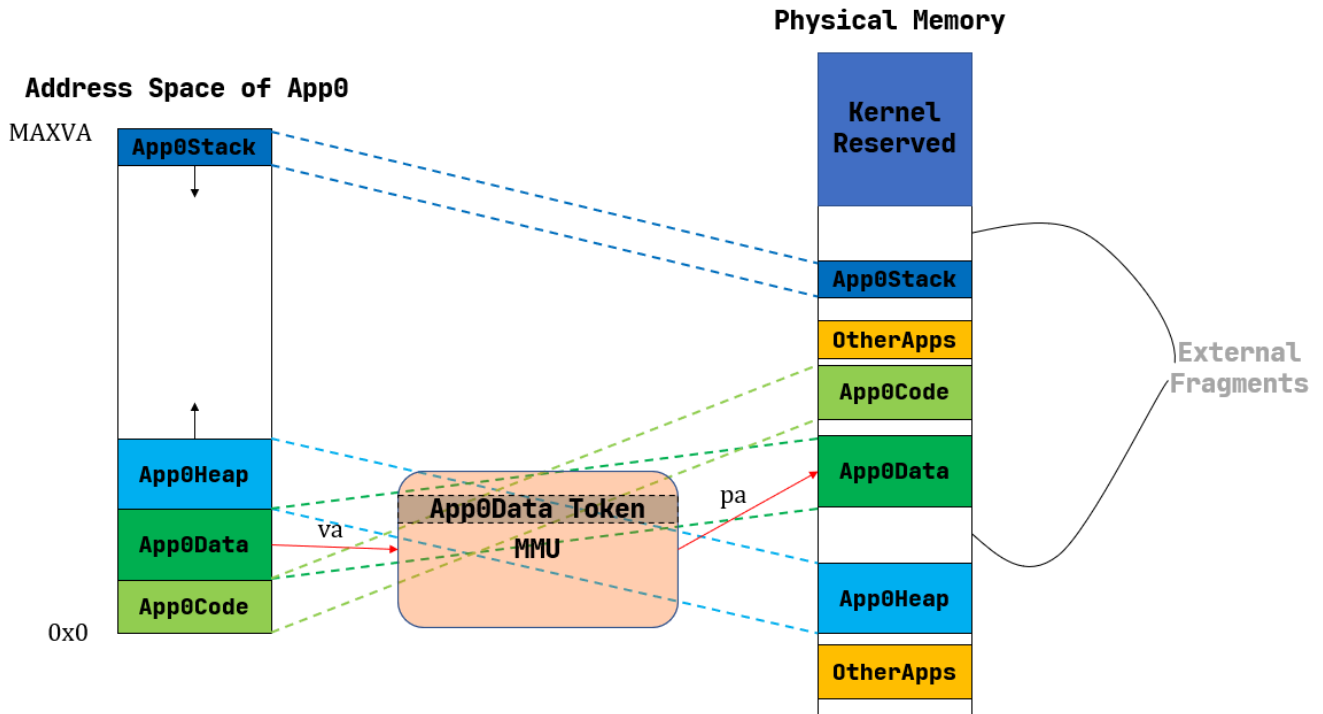
分页内存管理

简单来说，分段内存管理中段的大小不一致，是造成外部碎片的主要原因。在之前的方案中，我们把整个应用的地址空间连续地放在物理内存里。假如所有应用地址空间大小都相同，那么只需要用一个类似“位图”的数据结构来记录哪些“插槽”被占用了。这样一来，分配和回收内存的单位是固定大小的一块，自然就不会有外部碎片了。

不过这种做法不够灵活，因为分配的“块”太大，反而会在应用自己的地址空间内部造成内部碎片。

为了兼顾灵活性和简洁性，我们可以让内核以固定大小的“块”来存放应用地址空间的数据。这样内核就可以用简单的插槽方式来管理内存，分配算法简单，且不会产生外部碎片。同时，这个“块”要足够小，让块内没被利用的部分（也就是内部碎片）也尽量小，从而提高内存的利用率。

这就是我们即将介绍的分页内存管理。



如上图所示，内核*页为单位管理物理内存。每个应用的地址空间可以划分成多个虚拟页面，同样，物理内存也可以划分成多个物理页帧，它们的大小相同。每个虚拟页面里存放的数据，实际上都保存在某个物理页帧中。

和分段内存管理相比，分页的粒度更小。在分页机制下，应用地址空间里的每个逻辑段都由多个虚拟页面组成，每个虚拟页面在地址转换时使用独立的映射关系，而不是像分段那样整个逻辑段共用一个映射。

为了方便将虚拟页面转换成物理页帧，我们给每个虚拟页面和物理页帧编号，分别叫做虚拟页号和物理页号。

每个应用都有一个自己的页表，里面记录了该应用地址空间中每一个虚拟页面对应的物理页帧的位置，也就是数据实际存放在哪里。如果把页表看成一组键值对，那么键就是虚拟页号，值就是物理页号。

当地址转换单元进行地址转换时，它会：

1. 根据虚拟地址找到它属于哪个虚拟页面，并得到虚拟页号；
2. 查找当前应用的页表，根据虚拟页号找到对应的物理页号；
3. 将虚拟地址在页面内的偏移量加到物理页帧的起始地址上，得到最终要访问的物理地址。

页表中除了记录物理页号，还会包含一组保护位，用来限制应用对该块内存的访问方式。常见的保护位包括 **r**（可读）、**w**（可写）、**x**（可执行）。如果应用的操作违反了这些限制，就会触发异常并被内核捕获。通过合理设置保护位，可以检查出一些明显的错误，例如试图修改只读的代码段，或者试图执行数据段中的内容。

当一个应用的地址空间较大时，页表里的条目数量也会很多（每个虚拟页面对应一个条目，上图中已省略未使用的部分），页表的体积会变得非常大，无法像以前那样仅用几个寄存器就能保存。因此，页表通常作为内核管理的一种数据结构存放在内存中。但是，CPU 在执行过程中也需要访问页表来进行地址翻译，这就要求内核和硬件在页表的内存布局上保持一致的约定。

由于分页内存管理既简单又灵活，它逐渐成为主流的内存管理方式，RISC-V 架构也采用了这一策略。接下来，我们将基于这一机制，动手从物理内存中抽象出应用的地址空间。

C 的内存布局

在 `memory_layout.h` 之中我们展示了内存的布局。

```
// the kernel expects there to be RAM
// for use by the kernel and user pages
// from physical address 0x80000000 to PHYSTOP.
#define KERNBASE 0x80200000L
#define PHYSTOP (0x80000000 + 128*1024*1024) // 128M

// map the trampoline page to the highest address,
// in both user and kernel space.

// one beyond the highest possible virtual address.
// MAXVA is actually one bit less than the max allowed by
// Sv39, to avoid having to sign-extend virtual addresses
// that have the high bit set.
#define MAXVA (1L << (9 + 9 + 9 + 12 - 1))

#define USER_TOP (MAXVA)
#define TRAMPOLINE (USER_TOP - PGSIZE)
#define TRAPFRAME (TRAMPOLINE - PGSIZE)

#define USTACK_BOTTOM (0x0)
```

其中前两项在上一节已经介绍过了。下面的 `MAXVA` 其实并不是 SV39 中最大的虚拟地址(39位全为1)。我们设置它为 `(1<<38)` 是因为 VA 的高位填充是根据 39 位来进行的。为了方便全部填充 0，就不考虑大于 `(1<<38)` 的虚拟地址。在我们的框架之中，`va` 不可能大于 `MAXVA`。大家可以看到，我们指定 `TRAMPOLINE` 和 `TRAPFRAME` 在 `va` 的最高位，这是为什么呢？大家可以自行思考一下，我们将在下面解释。

注解

本节部分内容参考自 *Operating Systems: Three Easy Pieces* 教材的 13~16 小节。

SV39多级页表机制：内容介绍

本节导读

在上一节中，我们已经简单介绍了分页内存管理的基本策略。接下来，我们将基于 RV64 架构提供的 SV39 分页机制，实现内核中对应的软件支持。由于内容较多，我们将分成两节进行讲解。

本节主要讲解以下内容：

1. 在 RV64 架构下，虚拟地址与物理地址的访问属性（可读、可写、可执行等）；
2. 地址的组成结构（页号、帧号、偏移量等）；
3. 地址可访问的空间范围；
4. 以及操作系统如何进行页表管理。

虚拟地址和物理地址

内存控制相关的 CSR 寄存器

默认情况下，内存管理单元（MMU）是关闭的。此时，无论 CPU 处于哪个特权级，所有访存地址都会直接当作物理地址，交由内存控制单元访问物理内存。

我们可以通过修改 S 特权级下名为 `satp` 的控制状态寄存器（CSR）来开启分页模式。开启后：

- S 特权级和 U 特权级的访存地址会被视为虚拟地址，需经过 MMU 转换为物理地址，再访问物理内存；
- M 特权级的访存地址仍被视为物理地址。

下图展示了 RV64 架构下 `satp` 寄存器的字段分布：

（此处可配图：satp寄存器字段示意图）

当 `MODE` 字段设置为 0 时，所有访存地址均被视为物理地址；

设置为 8 时，则启用 SV39 分页机制。此时，所有 S/U 特权级的访存地址被视为 39 位虚拟地址，需先经过 MMU 的地址转换流程：

- 若转换成功，则得到一个 56 位的物理地址，并访问物理内存；
- 若转换失败，则会触发异常，从而体现该机制的内存保护能力。

虚拟地址和物理地址均为字节地址：

- 39 位虚拟地址理论上可访问最大 512 GiB 的地址空间；
- 56 位物理地址理论上可访问的物理内存更大。

但在实际中，无论是虚拟地址还是物理地址，只有其中很小一部分是真正有效且能通过 MMU 或内存控制单元检查的。因此，理论上的容量上限目前并无实际意义。

我们采用分页管理方式，将单个页面大小设为 4 KiB。每个虚拟页面和物理页帧均与页面大小对齐，即：

- 地址区间 [0, 4 KiB) 对应第 0 个虚拟页面/物理页帧；
- 地址区间 [4 KiB, 8 KiB) 对应第 1 个，依此类推。

4 KiB 需要用 12 位字节地址表示，因此虚拟地址和物理地址均分为两部分：

- 低 12 位 ([11:0]) 称为页内偏移，表示该地址在其所在页面内的相对位置；
- 虚拟地址的高 27 位 ([38:12]) 称为虚拟页号；
- 物理地址的高 44 位 ([55:12]) 称为物理页号。

页号用于确定一个地址属于哪一个虚拟页面或物理页帧。

地址转换以页为单位进行，转换前后，页内偏移部分保持不变。可以理解为：MMU 从虚拟地址中取出 27 位虚拟页号，通过查页表得到对应的物理页号（如果存在），然后将得到的 44 位物理页号与虚拟地址的 12 位页内偏移拼接，最终形成 56 位物理地址。

说明

为什么 RV64 架构中虚拟地址只有 39 位？

在 64 位架构上，虚拟地址长度应与位宽一致，即 64 位。但在 SV39 分页模式下，只有低 39 位是实际用于地址转换的。该模式规定，64 位虚拟地址的 [63:39] 这 25 位必须与第 38 位的值相同，否则 MMU 会直接判定其为非法虚拟地址。通过该检查后，MMU 才会取出低 39 位，尝试将其转换为 56 位的物理地址。

也就是说，在全部 2^{64} 个虚拟地址中，只有以下两部分能通过 MMU 检查：

- 最低的 256 GiB（当第 38 位为 0 时）；
- 最高的 256 GiB（当第 38 位为 1 时）。

在编写软件时，地址的位宽依然是 64 位，但我们需清楚只有上述两部分是实际可用的。虽然这两部分空间已经非常庞大，但本节重点在于介绍 MMU 的机制，因此我们强调：MMU 实际用于地址转换的虚拟地址只有 39 位。

多级页表原理

最简单的页表实现方式是线性表。也就是说，按照虚拟页号从 0 开始、地址从低到高的顺序，依次将每个虚拟页号对应的页表项存放在内存中（我们之前提到，页表太大，无法直接保存在 CPU 内部）。每个页表项的大小为 8 字节，因此，只要知道了第一个页表项（对应虚拟页号 0）存放的物理地址 `base_addr`，就可以直接计算出任何一个虚拟页号 `i` 对应的页表项位置。如下图所示：

具体来说，对于虚拟页号 `i`，如果某个应用的页表起始地址是 `base_addr`，那么这个页表项就存放在物理地址 `base_addr + 8 × i` 的位置。这种方式对 MMU（内存管理单元）的实现和内核的软件控制来说非常简单直接。

但遗憾的是，这种方式在物理内存上是行不通的。因为虚拟页号有 2^{27} 种可能，每个页表项占 8 字节，那么一个页表就需要占用 1 GiB 的内存！这还没算上应用本身的数据所占的内存。作为对比，我们使用的 K210 开发板总共只有 8 MiB 物理内存。所以，从内存占用的角度看，线性表这种实现方式是完全不可行的。

线性表的主要问题在于：它存储了所有虚拟页号（共 2^{27} 个）对应的页表项，但实际一个应用只会用到整个 512 GiB 地址空间中极小的一部分（本教程中的应用通常只使用几十到几百 KiB）。这就导致页表中绝大部分的表项（对应那些根本用不到的虚拟页号）都是浪费的。

那么，如何优化呢？核心思想是按需分配。也就是说，只维护那些真正被使用的虚拟页号的映射，并且只为此分配必要的内存。

我们可以这样理解：每个应用刚开始运行时，它的地址空间是“空”的，所有虚拟页号都是无效的。这时，页表本身可以不占用任何内存。当 MMU 进行地址转换时，遇到任何虚拟页号都直接判定为无效即可。之后，随着内核为应用的各个逻辑段（如代码段、数据段）分配好物理位置，并以虚拟页面为单位逐步建立映射，页表才会相应地被填充，占用的内存也会逐渐增加。

这种“按需分配”的思想在计算机科学中非常常见。为了帮助理解，我们来看一个类似的数据结构例子。

假设我们要维护一个字符串的多重集合（允许重复元素），集合中所有字符串都来自字符集 $\{a, b, c\}$ ，且长度固定为 n 。集合初始为空，支持两种操作：1) 插入一个字符串；2) 查询一个字符串出现的次数。

为了简化，假设 $n = 3$ 。我们可以使用一种叫做字典树 (Trie) 的数据结构，如下图所示：

字典树由多个节点（图中椭圆形）组成。每个节点代表一个可能的字符串前缀。每个节点内部只存储三个指针。

1. 非叶节点（图中蓝色）：它的三个指针各自指向一个子节点（分别对应下一个字符是 a, b 或 c ）。
2. 叶节点（图中绿色）：它的三个指针不再指向其他节点，而是直接存储一个长度为 n 的完整字符串的计数值。

对于插入或查询操作，我们只需根据字符串的每个字符，从根节点开始，沿着对应的指针一步步向下走，最终就能找到存储该字符串计数的位置（叶节点），然后进行更新或读取。

关键点在于：我们只为实际插入的字符串创建节点。如果某些字符串（或其前缀）从未被插入，那么对应的节点就根本不需要分配。

例如，开始时只有一个根节点。当我们插入字符串 "acb" 时，我们只需要为前缀 "a" 和 "ac" 创建新的节点。注意，"ac" 节点在这里就是叶节点，它的 'b' 指针不是指向另一个节点，而是直接存储字符串 "acb" 的计数。此时，我们无法访问到其他未创建的节点（比如根节点对应的 'b' 或 'c' 路径，或者 'a' 节点对应的 'a' 或 'b' 路径）。如果再插入另一个字符串，可能只需要再创建少数新节点，如果路径上已有节点，则可以复用。

这说明了：字典树占用的内存（节点数量），是随着插入的字符串数量线性增长的，而不是一开始就分配好所有可能节点。

你可能会好奇，为什么要花篇幅介绍字典树呢？因为 SV39 分页机制本质上就是一颗特殊的字典树。

在 SV39 中，一个 27 位的虚拟页号可以被看作一个长度为 3 的“字符串”，不过它的“字符集”非常大，是 $\{0, 1, 2, \dots, 511\}$ （因为每 9 个比特构成一个“字符”）。在这棵树里，我们不再存储字符串的计数，而是要找到这个“字符串”（虚拟页号）对应的最终页表项（里面包含物理页号）。

1. 叶节点：它需要存储 512 个 8 字节的页表项，总大小正好是 4KiB，可以完整地放入一个物理页帧中。
2. 非叶节点：从功能上说，它只需要存储 512 个指向下一级节点的指针。但在实现上，为了统一和简化，非叶节点也采用和叶节点相同的形式，存储 512 个页表项，同样占用一个 4KiB 的物理页帧。当我们想从一个非叶节点向下级走时，就查看当前“字符”（即虚拟页号中的 9 位）所对应的那个页表项，其中的“物理页号”字段就指向了下一级节点所在的物理页帧。

每个节点（物理页帧）内部，可以看作一个小的线性表：用节点的起始物理地址，加上“字符”值乘以 8（字节）的偏移量，就能找到对应的页表项。对于非叶节点，这个页表项是指针；对于叶节点，这个页表项就是最终用于地址转换的映射。

这种利用多级节点结构实现的页表，就叫做多级页表 (Multi-Level Page-Table)。由于 SV39 将 27 位虚拟页号分成了三段（每段 9 位），分别作为三级索引，所以它是一种三级页表。

需要注意的是，非叶节点中的页表项，其标志位的含义与叶节点中的有所不同：

1. 当 V (Valid) 位为 0 时，表示这是一个空指针，无法继续向下查找，对应的虚拟地址范围无效。
2. 当 V 位为 1，且 R/W/X 位全为 0 时，表示这是一个合法的页目录项。它包含的物理页号指针指向下一级页表（节点）。
3. 当 V 位为 1，且 R/W/X 位不全为 0 时，表示这是一个合法的页表项（叶节点项）。它包含了虚拟地址最终映射到的物理页号，并定义了该页的访问权限（读、写、执行）。

最后，附上 SV39 中 R/W/X 权限位的组合含义：

注解

大页 (Huge Page)

本教程中并没有用到大页的知识，这里只是作为拓展，不感兴趣的读者可以跳过。

事实上正确的说法应该是：只要 R/W/X 不全为 0 就会停下来，直接从当前的页表项中取出物理页号进行最终的地址转换。如果这一过程并没有发生在多级页表的最深层，那么在地址转换的时候并不是直接将物理页号和虚拟地址中的页内偏移接在一起得到物理地址，这样做会有问题：由于有若干级页索引并没有被使用到，即使两个虚拟地址的这些级页索引不同，还是会最终得到一个相同的物理地址，导致冲突。

我们需要重新理解将物理页号和页内偏移“接起来”这一行为，它的本质是将物理页号对应的物理页帧的起始物理地址和页内偏移进行求和，前者是将物理页号左移上页内偏移的位数得到，因此看上去恰好就是将物理页号和页内偏移接在一起。但是如果在从多级页表往下走的中途停止，未用到的页索引会和虚拟地址的 12 位页内偏移一起形成一个位数更多的页内偏移，也就对应于一个大页，在转换物理地址的时候，其算法仍是上述二者求和，但那时便不再是简单的拼接操作。

在 SV39 中，如果使用了一级页索引就停下来，则它可以涵盖虚拟页号的前 9 位为某一固定值的所有虚拟地址，对应于一个 1GiB 的大页；如果使用了二级页索引就停下来，则它可以涵盖虚拟页号的前 18 位为某一固定值的所有虚拟地址，对应于一个 2MiB 的大页。以同样的视角，如果使用了所有三级页索引才停下来，它可以涵盖虚拟页号为某一个固定值的所有虚拟地址，自然也就对应于一个大小为 4KiB 的虚拟页面。

使用大页的优点在于，当地址空间的大块连续区域的访问权限均相同的时候，可以直接映射一个大页，从时间上避免了大量页表项的索引和修改，从空间上降低了所需节点的数目。但是，从内存分配算法的角度，这需要内核支持从物理内存上分配三种不同大小的连续区域（4KiB 或是另外两种大页），便不能使用更为简单的插槽式管理。权衡利弊之后，本书全程只会以 4KiB 为单位进行页表映射而不会使用大页特性。

那么 SV39 多级页表相比线性表到底能节省多少内存呢？这里直接给出结论：设某个应用地址空间实际用到的区域总大小为 S 字节，则地址空间对应的多级页表消耗内存为 $\frac{S}{512}$ 左右。下面给出了详细分析，对此不感兴趣的读者可以直接跳过。

注解

分析 SV39 多级页表的内存占用

我们知道，多级页表的总内存消耗取决于节点的数目，每个节点则需要一个大小为 4KiB 物理页帧存放。不妨设某个应用地址空间中的实际用到的总空间大小为 S 字节，则多级页表所需的内存至少有这样两个上界：

1. 每映射一个 4KiB 的虚拟页面，最多需要新分配两个物理页帧来保存新的节点，加上初始就有一个根节点，因此消耗内存不超过 $4\text{KiB} \times (1 + 2 \frac{S}{4\text{KiB}}) = 4\text{KiB} + 2S$ ；
2. 考虑已经映射了很多虚拟页面，使得根节点的 512 个孩子节点都已经被分配的情况，此时最坏的情况是每次映射都需要分配一个不同的最深层节点，加上根节点的所有孩子节点并不一定都被分配，从这个角度来讲消耗内存不超过 $4\text{KiB} \times (1 + 512 + \frac{S}{4\text{KiB}}) = 4\text{KiB} + 2\text{MiB} + S$ 。

虽然这两个上限都可以通过刻意构造一种地址空间的使用来达到，但是它们看起来很不合理，因为它们均大于 S ，也就是元数据比数据还大。其实，真实环境中一般不会有如此极端的使用方式，更加贴近实际的是下面一种上限：即除了根节点的一个物理页帧之外，地址空间中的每个实际用到的大小为 T 字节的连续区间会让多级页表额外消耗不超过 $4\text{KiB} \times (\lceil \frac{T}{2\text{MiB}} \rceil + \lceil \frac{T}{1\text{GiB}} \rceil)$ 的内存。这是因为，括号中的两项分别对应为了映射这段连续区间所需要新分配的最深层和次深层节点的数目，前者每连续映射 2MiB 才会新分配一个，而后者每连续映射 1GiB 才会新分配一个。由于后者远小于前者，可以将后者忽略，最后得到的结果近似于 $\frac{T}{512}$ 。而一般情况下我们对于地址空间的使用方法都是在其中放置少数几个连续的逻辑段，因此当一个地址空间实际使用的区域大小总和为 S 字节的时候，我们可以认为为此多级页表消耗的内存存在 $\frac{S}{512}$ 左右。相比线性表固定消耗 1GiB 的内存，这已经相当可以接受了。

上面主要是对一个固定应用的多级页表进行了介绍。在一个多任务系统中，可能同时存在多个任务处于运行/就绪状态，它们的多级页表在内存中共存，那么 MMU 应该如何知道当前做地址转换的时候要查哪一个页表呢？回到 satp CSR 的布局，其中的 PPN 字段指的就是多级页表根节点所在的物理页号。因此，每个应用的地址空间就可以用包含了它多级页表根节点所在物理页号的 satp CSR 代表。在我们切换任务的时候，satp 也必须被同时切换。

最后的最后，我们给出 SV39 地址转换的全过程图示来结束多级页表原理的介绍：

编程作业：内存映射 (mmap/munmap)

在理解了地址空间和多级页表的原理后，本节将通过实现 `mmap` 和 `munmap` 系统调用来加深对虚拟内存管理的理解。

作业目标

实现两个内存映射相关的系统调用：

系统调用	调用号	功能
<code>sys_mmap</code>	222	将一段虚拟地址空间映射到物理内存（匿名映射）
<code>sys_munmap</code>	215	取消已有的内存映射

`sys_mmap` 接口定义：

```
/// 功能：申请一段虚拟地址空间并映射到物理内存
/// 参数：
///   start：起始虚拟地址，必须页对齐
///   len：映射长度（字节）
///   prot：权限位（bit0=读，bit1=写，bit2=执行）
///   flags：映射标志（本实现中忽略）
/// 返回值：成功返回 0，失败返回 -1
int sys_mmap(uint64 start, uint64 len, int prot, int flags);
```

`sys_munmap` 接口定义：

```
/// 功能：取消一段虚拟地址的映射
/// 参数：
///   start：起始虚拟地址，必须页对齐
///   len：取消映射的长度（字节）
/// 返回值：成功返回 0，失败返回 -1
int sys_munmap(uint64 start, uint64 len);
```

问题分析

问题1：mmap 的本质是什么？

回顾本章介绍的页表机制，mmap 的本质是建立虚拟地址到物理地址的映射关系。具体步骤：

- 1. 检查目标虚拟地址范围是否可用（没有被其他映射占用）
- 2. 分配物理页面
- 3. 在页表中建立映射

问题2：如何检查地址冲突？

使用 `walk()` 函数检查目标地址是否已被映射。`walk()` 的第三个参数 `alloc` 控制是否在 PTE 不存在时创建中间页表：

```
// 检查时传入 alloc=0，避免创建不必要的页表结构
pte_t *pte = walk(p->pagetable, va, 0);
if (pte != NULL && (*pte & PTE_V) != 0)
    return -1; // 已被映射
```

问题3：权限位如何转换？

mmap 的 `prot` 参数格式与页表的权限位格式不同，需要进行转换：

prot 位	含义	PTE 权限位
bit 0	可读	PTE_R (bit 1)
bit 1	可写	PTE_W (bit 2)
bit 2	可执行	PTE_X (bit 3)

```
int perm = PTE_U; /* 用户可访问 */
if (prot & 0x1) perm |= PTE_R;
if (prot & 0x2) perm |= PTE_W;
if (prot & 0x4) perm |= PTE_X;
```

代码实现

sys_mmap 实现

```
// os/syscall.c

/* ch4: sys_mmap - 匿名内存映射 */
int sys_mmap(uint64 start, uint64 len, int prot, int flags)
{
    struct proc *p = curr_proc();

    /* ch4: 检查起始地址页对齐 */
    if (!PGALIGNED(start))
        return -1;

    /* ch4: 检查prot有效性: 其他位必须为0 */
    if ((prot & ~0x7) != 0)
        return -1;

    /* ch4: 检查prot至少有一个权限(R/W/X) */
    if ((prot & 0x7) == 0)
        return -1;

    /* ch4: len为0时直接返回成功 */
    if (len == 0)
        return 0;

    /* ch4: 将len向上取整到页边界 */
    uint64 npages = (len + PGSIZE - 1) / PGSIZE;

    /* ch4: 检查[start, start+len)是否已被映射 */
    for (uint64 va = start; va < start + npages * PGSIZE; va += PGSIZE) {
        pte_t *pte = walk(p->pagetable, va, 0);
        if (pte != NULL && (*pte & PTE_V) != 0)
            return -1; /* 已被映射 */
    }
}
```

```

/* ch4: 将prot转换为PTE标志位 */
int perm = PTE_U; /* 用户可访问 */
if (prot & 0x1) perm |= PTE_R;
if (prot & 0x2) perm |= PTE_W;
if (prot & 0x4) perm |= PTE_X;

/* ch4: 逐页映射 */
for (uint64 i = 0; i < npages; i++) {
    uint64 va = start + i * PGSIZE;
    char *mem = kalloc();
    if (mem == NULL)
        return -1; /* 内存不足 */
    memset(mem, 0, PGSIZE);
    if (mappages(p->pagetable, va, PGSIZE, (uint64)mem, perm) != 0) {
        kfree(mem);
        return -1;
    }
}

return 0;
}

```

sys_munmap 实现

```

// os/syscall.c

/* ch4: sys_munmap - 取消内存映射 */
int sys_munmap(uint64 start, uint64 len)
{
    struct proc *p = curr_proc();

    /* ch4: 检查起始地址页对齐 */
    if (!PGALIGNED(start))
        return -1;

    /* ch4: len为0时直接返回成功 */
    if (len == 0)
        return 0;

    /* ch4: 将len向上取整到页边界 */
    uint64 npages = (len + PGSIZE - 1) / PGSIZE;

    /* ch4: 检查[start, start+len)是否全部已映射 */
    for (uint64 va = start; va < start + npages * PGSIZE; va += PGSIZE) {
        pte_t *pte = walk(p->pagetable, va, 0);
        if (pte == NULL || (*pte & PTE_V) == 0)
            return -1; /* 未映射 */
    }

    /* ch4: 取消映射并释放物理内存 */
    uvmunmap(p->pagetable, start, npages, 1);
}

```



```
    return 0;
}
```

遇到的问题与解决

问题：进程退出时的 freewalk panic

现象：运行 mmap 测试时，进程退出后出现 `freewalk: leaf panic`。

分析：

1. mmap 可以在任意地址分配内存（只要没被占用）
2. 进程的 `max_page` 字段只记录程序代码和栈的范围
3. `uvmfree` 只清理 0 到 `max_page` 范围的页面
4. mmap 分配的高地址页面没有被清理，导致 `freewalk` 遇到"意外"的叶子页面

解决方案：修改 `freewalk` 函数，遇到叶子页面时释放它而不是 panic：

```
// os/vm.c

/* ch4: 修改为同时释放叶子页面，支持mmap区域的清理 */
void freewalk(pagetable_t pagetable)
{
    for (int i = 0; i < 512; i++) {
        pte_t pte = pagetable[i];
        if ((pte & PTE_V) && (pte & (PTE_R | PTE_W | PTE_X)) == 0) {
            // 指向下级页表
            uint64 child = PTE2PA(pte);
            freewalk((pagetable_t)child);
            pagetable[i] = 0;
        } else if (pte & PTE_V) {
            /* ch4: 释放叶子页面的物理内存 */
            uint64 pa = PTE2PA(pte);
            kfree((void *)pa);
            pagetable[i] = 0;
        }
    }
    kfree((void *)pagetable);
}
```

与第三章的关联：完善 sys_trace 权限检查

在实现 mmap 的过程中，我们发现第三章的 `sys_trace` 存在安全隐患。mmap 可以创建只读或只写的页面，但原来的 `sys_trace` 没有检查权限。

改进：读操作检查 PTE_R，写操作检查 PTE_W：

```
/* ch4: 写操作 - 检查地址是否用户可见且可写 */
pte_t *pte = walk(p->pagetable, id, 0);
if (pte == NULL || (*pte & PTE_V) == 0 ||
    (*pte & PTE_U) == 0 || (*pte & PTE_W) == 0)
    return -1;
```

知识点总结

通过本作业，我们深入理解了以下概念：

概念	说明
页表操作	mappings() 建立映射， unmap() 取消映射， walk() 查找 PTE
权限位	PTE 中的 V/R/W/X/U 位控制页面的有效性和访问权限
内存分配	kalloc() 分配物理页， kfree() 释放，需配对使用
资源清理	进程退出时必须释放所有分配的资源，包括 mmap 区域

修改文件清单

文件	修改内容
os/syscall.c	实现 sys_mmap() 和 sys_munmap()，完善 sys_trace() 权限检查
os/vm.c	修改 freewalk() 以支持 mmap 区域的清理